# Module 13 – DevOps and CI/CD

## 1. Introduction to DevOps

### What is DevOps?

**DevOps** is a software development methodology that combines **Development (Dev)** and **Operations (Ops)** to improve collaboration, automate workflows, and deliver software faster and more reliably.

Traditionally, development and operations teams worked independently. Developers focused on building applications, while operations teams managed deployment and maintenance. This separation often led to delays, communication gaps, and deployment issues.

DevOps bridges this gap by encouraging collaboration, automation, and continuous feedback throughout the software development lifecycle (SDLC).

### Traditional Development vs DevOps

| Traditional Approach | DevOps Approach |
|---|---|
| Development and Operations work separately | Development and Operations collaborate throughout the project |
| Manual software deployment | Automated build, testing, and deployment |
| Infrequent software releases | Frequent and continuous releases |
| Slow feedback cycle | Continuous monitoring and rapid feedback |
| Higher deployment risk | Faster and more reliable deployments |

### Goals of DevOps

The primary goals of DevOps are:

- Deliver software faster and more frequently.
- Improve collaboration between development and operations teams.
- Automate repetitive tasks such as building, testing, and deployment.
- Improve software quality through continuous testing.
- Reduce deployment failures.
- Enable quick recovery from failures.
- Provide continuous monitoring and feedback.

### Benefits of DevOps

#### Faster Software Delivery

Automation allows new features and updates to be released more quickly.

**Improved Collaboration**

Developers, testers, and operations engineers work together, reducing communication barriers.

**Higher Software Quality**

Continuous testing helps identify defects early in the development process.

**Increased Reliability**

Automated deployments reduce human errors and ensure consistent releases.

**Faster Issue Resolution**

Continuous monitoring enables teams to detect and fix problems quickly.

**Scalability**

Automated infrastructure and deployment processes make it easier to scale applications.

**Better Customer Satisfaction**

Frequent releases and faster bug fixes improve the overall user experience.

**Key DevOps Practices**

**Continuous Integration (CI)**

Developers frequently merge their code into a shared repository where automated builds and tests are executed.

**Continuous Delivery (CD)**

Software is automatically prepared for release after successful testing. Deployment to production requires manual approval.

**Continuous Deployment**

Every successfully tested change is automatically deployed to production without manual intervention.

**Infrastructure as Code (IaC)**

Infrastructure (servers, networks, storage, etc.) is managed using code instead of manual configuration.

Examples:

- Terraform
- AWS CloudFormation
- Ansible

**Continuous Monitoring**

Applications are continuously monitored to detect performance issues, failures, and security threats.

Examples:

- Prometheus
- Grafana
- ELK Stack

**Automation**

Routine tasks such as testing, deployment, infrastructure provisioning, and monitoring are automated to improve efficiency and consistency.

**DevOps Lifecycle**

The DevOps lifecycle consists of the following stages:

1. **Plan** – Define project requirements and tasks.
2. **Develop** – Write and manage source code.
3. **Build** – Compile and package the application.
4. **Test** – Perform automated testing.
5. **Release** – Prepare the application for deployment.
6. **Deploy** – Deploy the application to the target environment.
7. **Operate** – Run and maintain the application.
8. **Monitor** – Continuously monitor application performance and collect feedback.

This cycle repeats continuously, enabling rapid and reliable software delivery.

# 2. Understanding CI/CD

### What is Continuous Integration (CI)?

**Continuous Integration (CI)** is a DevOps practice in which developers frequently integrate their code changes into a shared repository.

Each integration triggers an automated pipeline that typically performs:

- Source code compilation.
- Dependency installation.
- Automated unit testing.
- Static code analysis.
- Build verification.

The goal is to detect integration issues early, before they become larger problems.

### CI Workflow

A typical Continuous Integration workflow is:

1. Developer writes code.
2. Changes are committed to the Git repository.
3. CI server detects the new commit.
4. The application is built automatically.
5. Automated tests are executed.
6. Results are reported to the development team.
7. Developers fix issues if any stage fails.

This process is repeated for every code change.

### What is Continuous Delivery (CD)?

**Continuous Delivery** extends Continuous Integration by ensuring that every successful build is automatically prepared for deployment.

After all tests pass:

- The application is packaged.
- Deployment artifacts are generated.
- The application is ready for production.

However, deployment to production requires **manual approval**.

### What is Continuous Deployment?

**Continuous Deployment** goes one step further than Continuous Delivery.

After successful testing:

- The application is automatically deployed to production.
- No manual approval is required.

This enables organizations to release software multiple times a day with minimal human intervention.

**CI vs Continuous Delivery vs Continuous Deployment**

| **Feature** | **Continuous Integration (CI)** | **Continuous Delivery** | **Continuous Deployment** |
|---|---|---|---|
| Code Integration | ✔ | ✔ | ✔ |
| Automated Build | ✔ | ✔ | ✔ |
| Automated Testing | ✔ | ✔ | ✔ |
| Automatic Deployment to Staging | ✘ | ✔ | ✔ |
| Automatic Deployment to Production | ✘ | ✘ (Manual approval required) | ✔ |
| Human Approval | Not applicable | Required before production | Not required |

**Benefits of CI/CD**

**Early Bug Detection**

Automated testing identifies defects soon after code is committed.

**Faster Development**

Developers receive rapid feedback, allowing issues to be fixed quickly.

**Reduced Deployment Risk**

Frequent, smaller deployments are easier to validate and troubleshoot.

**Improved Code Quality**

Continuous testing and static analysis help maintain high coding standards.

**Faster Release Cycles**

Applications can be released more frequently with greater confidence.

**Greater Reliability**

Automated pipelines ensure that every deployment follows the same validated process.

**Increased Productivity**

Developers spend less time on manual builds and deployments, allowing them to focus on feature development.

**Typical CI/CD Pipeline**

A standard CI/CD pipeline includes the following stages:

1. **Code** – Developers commit changes to a version control system (e.g., Git).
2. **Build** – The application is compiled and packaged.
3. **Test** – Automated unit, integration, and quality checks are performed.
4. **Release** – A release-ready artifact is generated.
5. **Deploy** – The application is deployed to staging or production.
6. **Monitor** – Application health and performance are continuously monitored.

Each stage is automated to ensure consistency and reliability.

**3. CI/CD Tools and Platforms**

Various tools are available to automate different stages of the CI/CD pipeline. Some of the most widely used platforms are described below.

**Jenkins**

**Jenkins** is one of the most popular open-source automation servers used to implement CI/CD pipelines.

**Key Features**

- Open-source and highly customizable.
- Supports thousands of plugins.
- Automates build, test, and deployment processes.
- Integrates with Git, Maven, Docker, Kubernetes, SonarQube, and many other tools.
- Suitable for both small and enterprise-scale projects.

**Common Use Cases**

- Automated software builds.

- Continuous Integration pipelines.
- Automated testing.
- Deployment automation.

**GitHub Actions**

**GitHub Actions** is GitHub's built-in CI/CD platform that allows automation directly within GitHub repositories.

**Key Features**

- Native integration with GitHub.
- Workflow automation using YAML configuration files.
- Automatic execution on repository events such as commits, pull requests, or releases.
- Supports Linux, Windows, and macOS runners.
- Easy integration with cloud platforms and container services.

**Common Use Cases**

- Build automation.
- Running unit tests.
- Code quality analysis.
- Automated deployments.
- Pull request validation.

**GitLab CI/CD**

**GitLab CI/CD** is the integrated CI/CD solution provided by GitLab.

**Key Features**

- Built directly into GitLab.
- Pipeline configuration through .gitlab-ci.yml.
- Automated builds, testing, security scanning, and deployments.
- Supports containerized applications and Kubernetes deployments.
- Provides integrated monitoring and reporting.

**Common Use Cases**

- Complete DevOps lifecycle management.
- Secure software delivery.
- Enterprise CI/CD pipelines.

**CircleCI**

**CircleCI** is a cloud-based CI/CD platform focused on fast and efficient build automation.

**Key Features**

- Cloud-native architecture.
- Parallel execution for faster builds.
- Easy integration with GitHub and Bitbucket.
- Supports Docker and Kubernetes.
- Scalable for projects of various sizes.

**Common Use Cases**

- Continuous Integration.
- Automated testing.
- Continuous deployment.
- Cloud-native application development.

**Comparison of Popular CI/CD Tools**

| **Feature** | **Jenkins** | **GitHub Actions** | **GitLab CI/CD** | **CircleCI** |
|---|---|---|---|---|
| Type | Open-source | GitHub integrated | GitLab integrated | Cloud-based |
| Installation | Self-hosted | Built into GitHub | Built into GitLab | Cloud service |
| Configuration | Jenkinsfile | YAML Workflow | .gitlab-ci.yml | YAML Configuration |
| Plugin Support | Extensive | Marketplace Actions | Built-in integrations | Built-in integrations |
| Ease of Setup | Moderate | Easy | Easy | Easy |
| Best Suited For | Enterprise and highly customized pipelines | GitHub-hosted projects | GitLab-hosted projects | Cloud-native and fast build pipelines |

**Best Practices for DevOps and CI/CD**

- Commit small, frequent code changes.
- Automate builds, testing, and deployments wherever possible.
- Use version control (Git) for all application code and infrastructure configurations.
- Perform automated testing at every stage of the pipeline.
- Integrate static code analysis tools (e.g., SonarQube) into CI pipelines.
- Monitor applications continuously after deployment.
- Use Infrastructure as Code (IaC) for consistent environment provisioning.
- Secure CI/CD pipelines by managing secrets and credentials appropriately.

- Regularly review pipeline performance and optimize build times.

**Quick Revision Points**

- **DevOps** combines **Development (Dev)** and **Operations (Ops)** to improve collaboration, automate workflows, and accelerate software delivery.
- Key DevOps practices include **Continuous Integration (CI)**, **Continuous Delivery (CD)**, **Continuous Deployment**, **Infrastructure as Code (IaC)**, **Continuous Monitoring**, and **Automation**.
- **Continuous Integration (CI)** involves frequently merging code into a shared repository, followed by automated builds and tests.
- **Continuous Delivery** ensures that every successful build is deployment-ready but requires manual approval before production deployment.
- **Continuous Deployment** automatically deploys every successfully tested change to production without manual intervention.
- A typical **CI/CD pipeline** consists of the stages: **Code → Build → Test → Release → Deploy → Monitor**.
- Popular CI/CD platforms include **Jenkins**, **GitHub Actions**, **GitLab CI/CD**, and **CircleCI**, each offering workflow automation, build orchestration, testing, and deployment capabilities.
- CI/CD improves software quality by enabling early bug detection, reducing deployment risk, increasing development speed, and ensuring consistent, reliable releases.